

창원댁 — 결과보고서

AI NATIVE ADVANCED · 개인 미션 (A1-3)
AI 웹 개발: 내 아이디어를 현실로, AI 웹 서비스 빌딩

항목	내용
과제명	AI 웹 개발: 내 아이디어를 현실로, AI 웹 서비스 빌딩 (미션과제 A1-3)
서비스명	창원댁
배포 URL	https://changwondaek.vercel.app
GitHub	https://github.com/iiiiiiii4319/A1-3
작성 형태	개인 프로젝트 (1인 수행)
핵심 기능	구 선택 → 국토부 실거래가 6개월 분석 → AI 요약 코멘트 생성
보너스 과제	방문자 분석 (Vercel Web Analytics)

목차

- I. 프로젝트 개요
- II. 서비스 구성 및 최종 결과물
- III. 핵심 기능 — AI 시장분석 (동작 결과)
- IV. 반응형 및 배포 공유
- V. 보너스 과제 — 방문자 분석
- VI. 개발 과정 요약 (AI 코딩 도구: Claude)
- VII. 기술적 이슈 및 해결
- VIII. AI 사전평가 결과
- IX. 자체 점검 체크리스트
- X. 최종 제출물 (필수 5종)
- X I. 결론 및 배운 점

I. 프로젝트 개요

창원에서 내 집 마련이나 이사를 계획하는 사람들은 "지금이 살 때가인가"를 고민하지만, 판단 근거는 대부분 중개업소 설명이나 커뮤니티 여론에 의존해 객관성이 떨어진다.

창원댁은 국토교통부가 공개하는 아파트 매매 실거래가 데이터를 사용해, 창원 5개 구(의창·성산·마산합포·마산회원·진해)의 최근 6개월 매매가 흐름을 계산하고, 이를 **AI가 이해하기 쉬운 한두 문장**으로 요약해 주는 웹서비스이다.

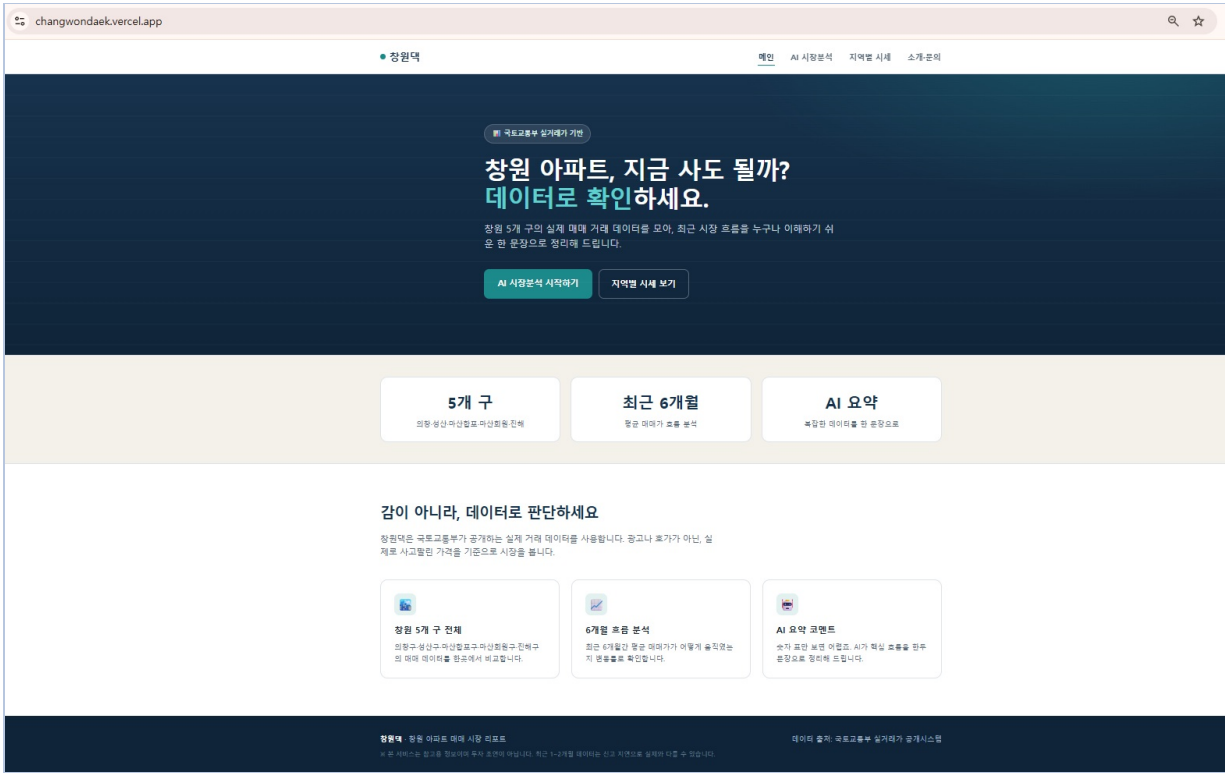
항목	내용
타겟 사용자	창원에서 내 집 마련 또는 이사를 계획 중인 실수요자
데이터 범위	아파트 매매 실거래가 · 5개 구 전체 · 최근 6개월 평균가 비교
기술 스택	HTML/CSS/JS (바닐라), Vercel Serverless Functions(Python), OpenAI API, 국토부 실거래가 API
배포	GitHub + Vercel (Web Analytics 포함)

II. 서비스 구성 및 최종 결과물

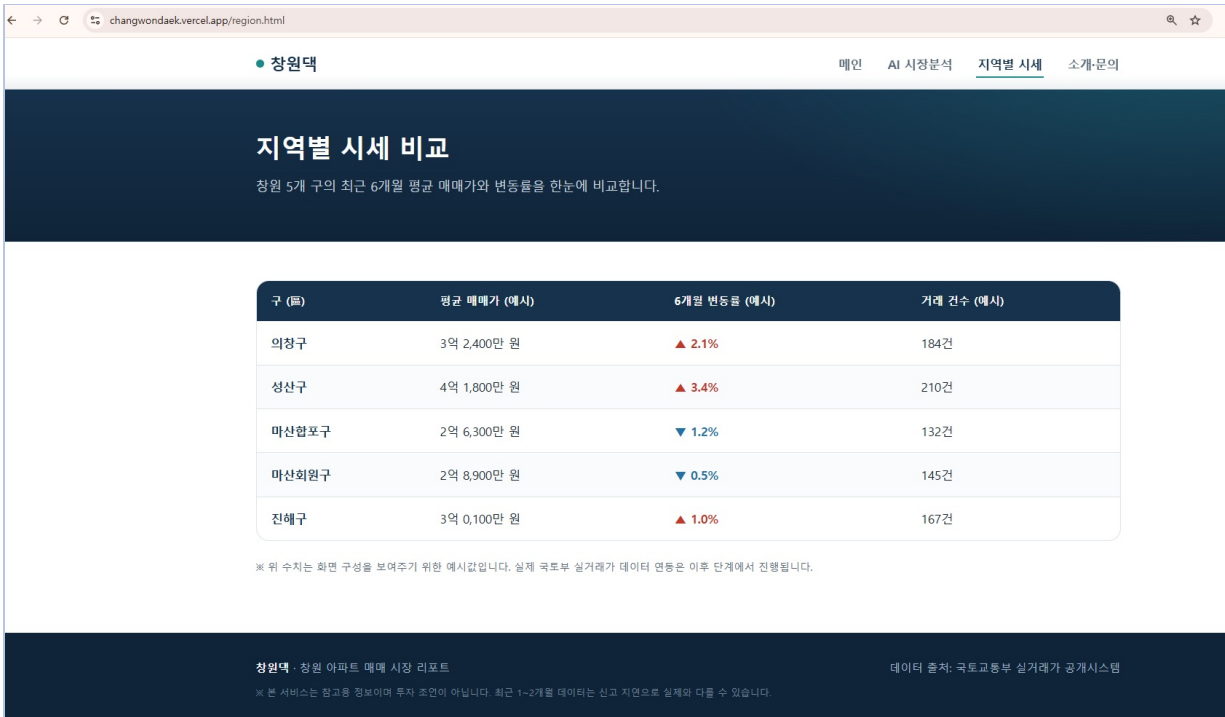
최소 3개 이상 요건을 충족하여 4개 페이지로 구성하였으며, 상단 네비게이션으로 상호 이동한다.

페이지	내용
메인	서비스 소개, 핵심 가치 제안, 주요 페이지 이동
AI 시장분석	구 선택 → 최근 6개월 평균 매매가 조회 → AI 요약 코멘트 (핵심 기능)
지역별 시세	창원 5개 구 시세 비교 표
소개·문의	서비스 취지, 데이터 출처, 면책 문구, 문의

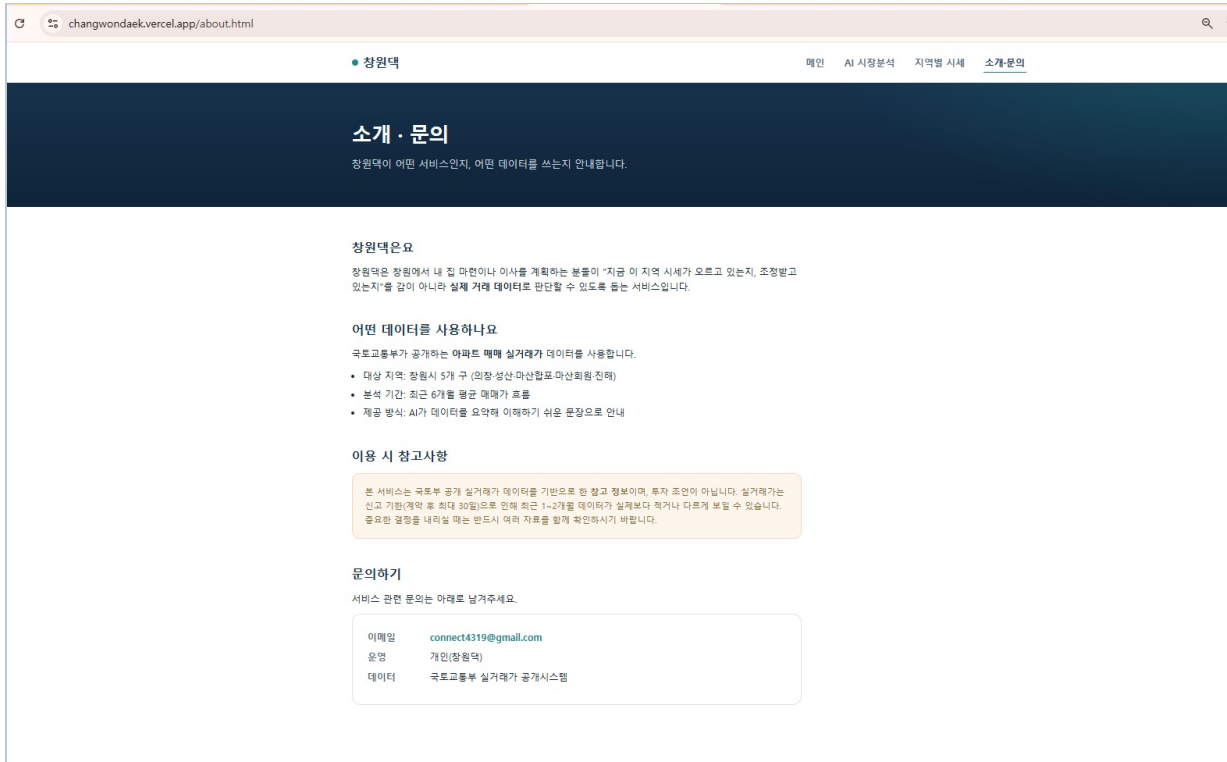
[그림 1] 메인 페이지 — 배포 URL(changwondaek.vercel.app)에서 동작



[그림 2] 지역별 시세 비교 페이지



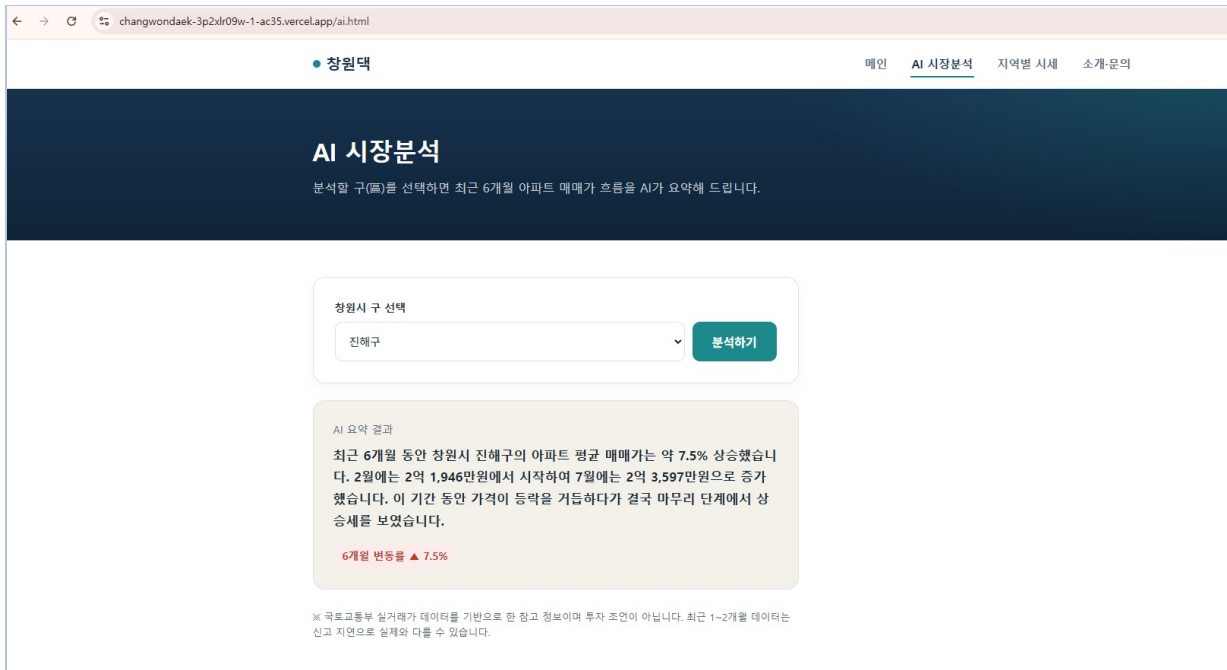
[그림 3] 소개·문의 페이지



III. 핵심 기능 — AI 시장분석 (동작 결과)

사용자가 구를 선택하면 백엔드(`api/analyze.py`)가 국토부 실거래가 API로 최근 6개월 데이터를 조회하고, 월별 평균 매매가와 변동률을 계산한 뒤, 그 수치를 OpenAI API에 전달하여 자연어 요약을 생성한다.

[그림 4] AI 시장분석 결과 — 진해구 6개월 변동률 +7.5% 요약 생성



백엔드 API는 아래와 같이 JSON 형태로 응답한다. (예: 성산구 — 월별 평균가, 변동률, AI 요약 포함)

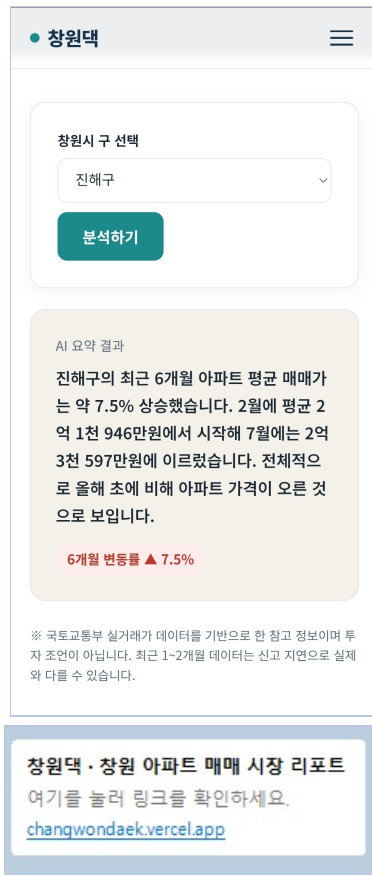
[그림 5] 백엔드 API 응답(JSON)



IV. 반응형 및 배포 공유

모바일/데스크톱에서 레이아웃이 깨지지 않도록 구현하였으며, 실제 휴대폰에서도 정상 동작을 확인했다. 또한 공개 URL을 통해 제3자가 접속·테스트할 수 있다.

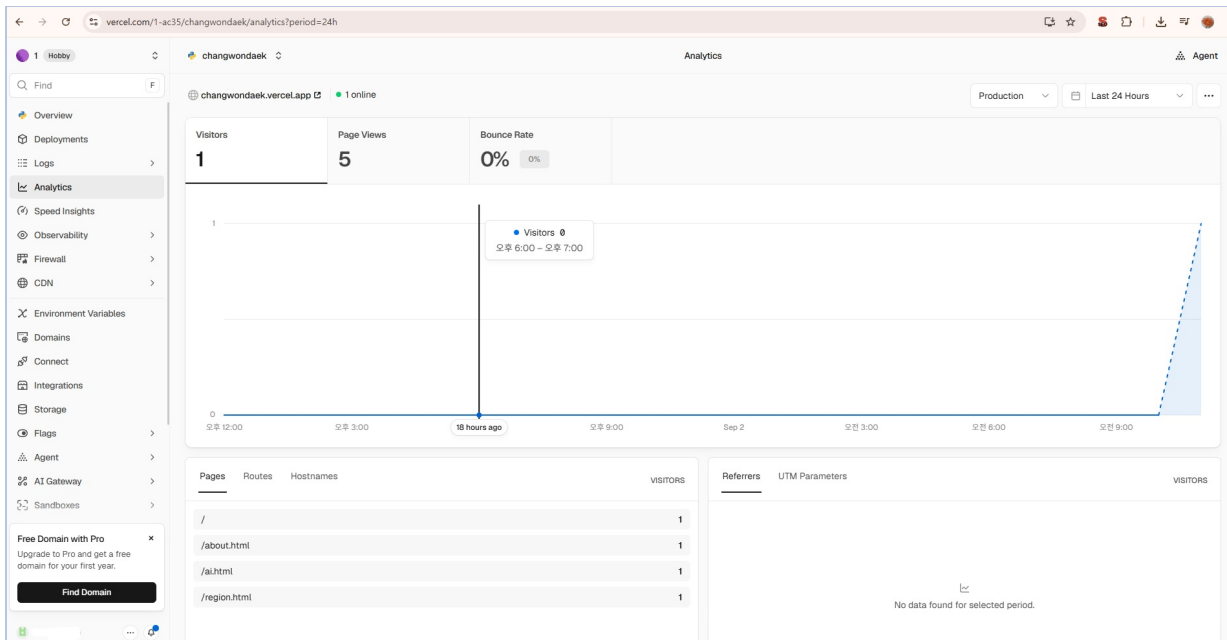
[그림 6] (좌) 모바일 반응형 화면 · (우) 카카오톡 링크 공유 미리보기



V. 보너스 과제 — 방문자 분석

Vercel Web Analytics를 적용하여 방문자 수와 페이지별 조회수를 확인할 수 있도록 하였다. 4개 페이지에 분석 스크립트를 삽입하고 배포하여, 방문 시 대시보드에 기록된다.

[그림 7] Vercel 방문자 분석 대시보드 — 페이지별 조회수 기록

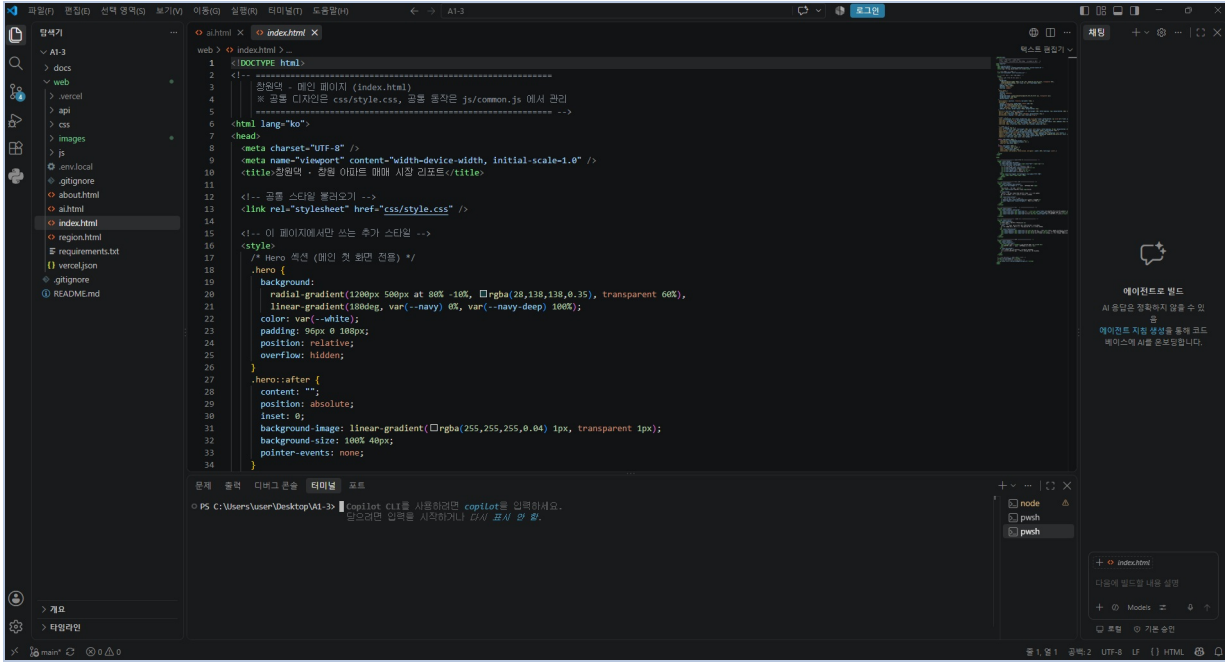


VI. 개발 과정 요약 (AI 코딩 도구: Claude)

AI 코딩 도구로는 **Claude**를 사용하였다. Claude를 활용해 코드를 작성하되, 오류가 발생하면 원인을 직접 파악·수정하며 단계적으로 진행하였다.

단계	수행 내용
기획	서비스 아이디어·타겟·페이지·AI 기능 정의, 기획서 작성
환경 구성	프로젝트 폴더 구조화, GitHub 저장소 생성, .gitignore·env.local 보안 세팅
프론트엔드	바닐라 HTML/CSS/JS로 4개 페이지 + 네비게이션 + 반응형 구현
데이터 연동	국토부 실거래가 API 호출 → 6개월 평균가·변동률 계산 (로컬 검증)
AI 연동	계산 결과를 OpenAI API로 전달해 자연어 요약 생성
백엔드	api/analyze.py 작성 (Vercel Serverless Function), fetch 연동, 실패 처리
배포	GitHub-Vercel 연동, 환경변수 등록, 라우팅 설정(vercel.json), 재배포
보너스·문서	방문자 분석 적용, README·결과보고서·증빙 정리

[그림 8] VS Code 작업 화면 — AI 코딩 도구를 활용한 개발 과정



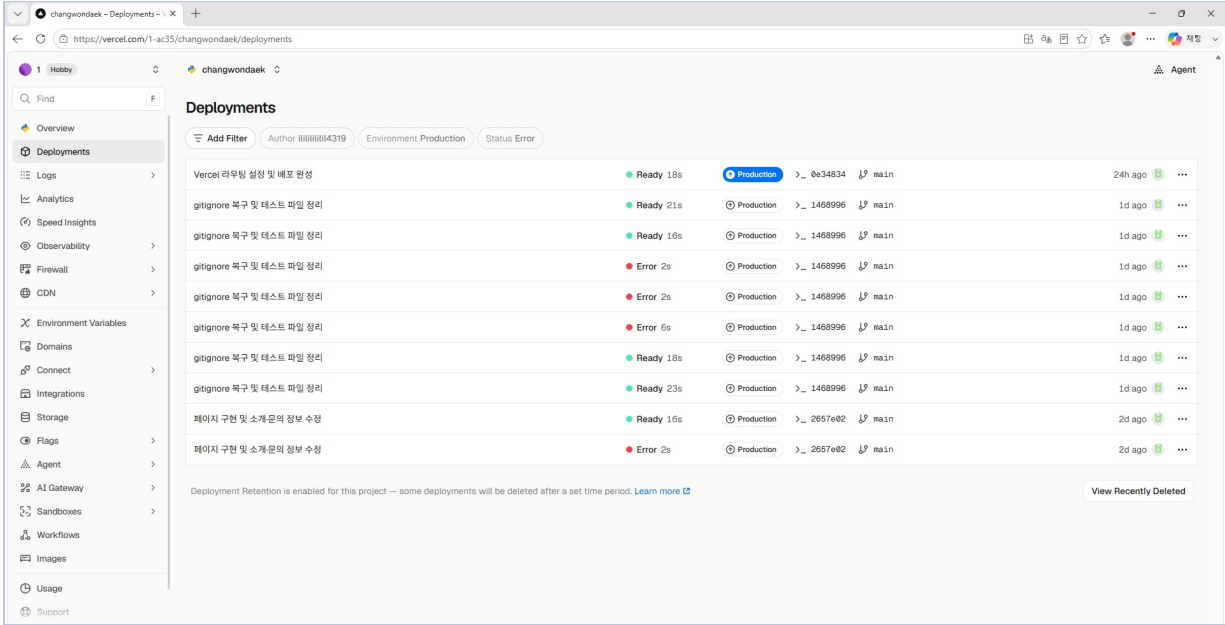
Ⅶ. 기술적 이슈 및 해결

개발·배포 과정에서 실제로 마주한 문제와 해결 과정을 기록한다. AI가 생성한 코드라도 오류 원인을 직접 파악하고 수정하는 것을 목표로 하였다.

이슈	원인	해결
OpenAI 인증 401 오류	OpenAI 정식 키가 아닌 다른 키 사용	platform.openai.com에서 정식 키 재발급 후 교체
배포 후 '데이터 부족' 오류	Vercel에 국토부 API 주소(MOLIT_API_URL) 미등록	환경변수 3개를 모두 등록 후 재배포
/ai.html이 JSON으로 표시	파이썬 함수가 모든 요청을 가로챈(라우팅 충돌)	vercel.json으로 정적/함수 경로를 명확히 분리
진입점(entrypoint) 빌드 오류	pyproject.toml과 requirements.txt 충돌	pyproject.toml 제거, requirements.txt만 유지
공유 링크 접속 불가	임시 배포 URL이 보호(Protected) 상태	고정 공개 URL(changwondaek.vercel.app) 사용

위 이슈들은 한 번에 해결된 것이 아니라, **Vercel 배포 로그에서 실패(Error) 원인을 확인 → 코드·설정 수정 → 재배포** 과정을 여러 차례 반복하며 해결하였다. 아래 배포 이력에서 여러 번의 **Error** 와 최종 **Ready** (Production) 상태가 이를 보여준다. 로컬에서는 확인하기 어려운 배포 환경 문제(환경변수, 라우팅, 빌드 설정 등)를 배포 로그를 근거로 직접 진단하고 재배포로 해결하였다.

[그림 8-1] Vercel 배포 이력 — 실패(Error) 진단 후 재배포로 최종 Ready(Production) 달성



Ⅷ. AI 사전평가 결과

네이토(Nato) AI 사전평가를 실시하여 구현·문서화 수준을 점검하였다. (평가 시각: 2026-09-02 17:49)

1) 최초 사전평가 결과

종합 점수	결과
88%	17개 항목 중 15개 통과 (PASS 15 / FAIL 2)

종합 평가: 배포 URL·다수 페이지·반응형·AI 입력→서버→화면 흐름·환경변수 사용 등 핵심 구현이 충실하다. 다만 테스트 입력 케이스(긴 입력) 처리와 응답 지연 개선(캐시·경량화 등)의 문서화가 누락되어 보완이 필요하다.

보완이 필요한 항목 (FAIL 2건)

항목	부족한 점	보완 방향
#4 긴 입력 처리	긴 입력(초과 길이) 케이스 처리 미구현 — 검증·잘림·에러 대응 누락	입력 길이/유효성 검증과 관련 안내 문구를 추가·문서화
#16 응답 지연 개선	응답 지연 개선 옵션(캐시/모델 경량화/요약 등) 문서 미제공	캐시 전략·경량 모델 대안·요약 전처리 등 개선 옵션을 문서로 정리

통과 항목 요약 (PASS 15건)

영역	통과 내용
배포·구성	배포 URL과 4개 페이지 구성 명확(#1), 프론트·백엔드 분리 구조와 역할 설명(#8)
반응형	반응형 미디어쿼리 및 모바일 확인 증빙 존재(#2)
AI 기능	입력 UI·fetch 호출·결과 표시 구현(#3), 로딩/성공/실패 상태 처리(#9), AI 도입 목적·프롬프트 제시(#15)
실패 처리	빈 입력·지연·API 오류 상황별 안내 메시지 구현(#5)
백엔드	환경변수로 키 로드(#6), 입력검증·JSON 응답 포맷 구현(#10), fetch·핸들러 경로 일치(#13)
보안·운영	키 비하드코딩·환경별 변수 등록 문서화(#14), 확장·키유출 대비 언급(#17)
문서화	제출 패키지 항목 명시(#7), 배포 문제 진단·재배포 절차 기록(#11), HTML/CSS/JS 역할·흐름 설명(#12)

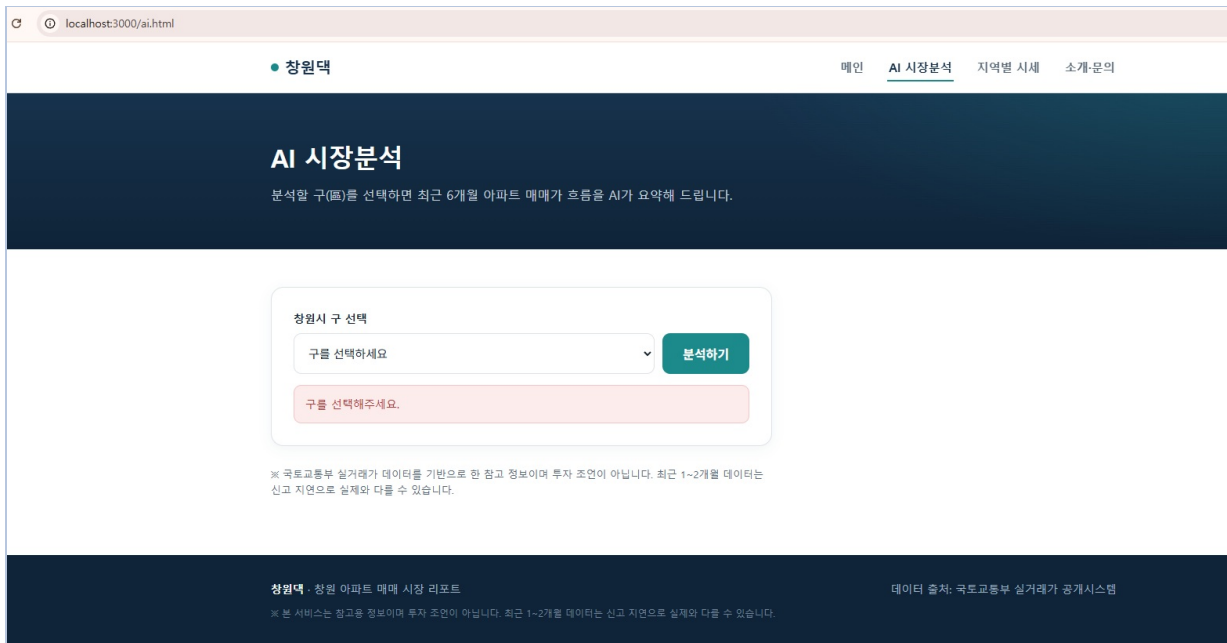
2) 보완 조치 및 실제 동작 재검증 (2026-09-02)

FAIL 2건을 문서 정리로만 끝내지 않고, `api/analyze.py` 에 실제 코드를 추가한 뒤 로컬 개발 서버(`vercel dev`)에서 직접 호출해 동작을 재검증하였다.

① #4 입력 검증 보완

`gu` 파라미터는 프론트에서 `<select>` 드롭다운으로만 값이 들어오지만, `/api/analyze` 는 누구나 쿼리스트링으로 직접 호출할 수 있으므로 백엔드에도 검증 로직(`validate_gu()`)을 추가하였다. ① 빈 값/공백 ② 비정상적으로 긴 입력 ③ 창원 5개 구 화이트리스트 미포함 값, 3단계로 걸러낸다.

[그림 9] 프론트엔드 — 빈 입력 시 안내 메시지



[그림 10] 백엔드 — 빈 값/목록에 없는 값 직접 호출 검증

```
>> # @ 정상 입력 첫 호출 - "cached":false 나와야 정상 (몇 초 걸림)
>> curl.exe "http://localhost:3000/api/analyze?gu=성산구"
>>
>> # @ 같은 구 즉시 재호출 - "cached":true 나와야 정상 (거의 즉시 응답)
>> curl.exe "http://localhost:3000/api/analyze?gu=성산구"
{"error": "구를 선택해주세요."}
PS C:\Users\user\Desktop\A1-3> curl.exe "http://localhost:3000/api/analyze?gu="
{"error": "구를 선택해주세요."}
PS C:\Users\user\Desktop\A1-3> curl.exe "http://localhost:3000/api/analyze?gu=%EC%9D%B4%EC%83%81%ED%95%9C%EA%B5%AC"
{"error": "지원하지 않는 지역입니다. 창원시 5개 구 중에서 선택해주세요."}
PS C:\Users\user\Desktop\A1-3> █
```

```
PS> curl.exe "http://localhost:3000/api/analyze?gu="
{"error": "구를 선택해주세요."}

PS> curl.exe "http://localhost:3000/api/analyze?gu=%EC%9D%B4%EC%83%81%ED%95%9C%EA%B5%AC"
{"error": "지원하지 않는 지역입니다. 창원시 5개 구 중에서 선택해주세요."}
```

② #16 응답 지연 개선(캐시) 보완

동일한 구를 짧은 시간(10분 TTL) 내에 다시 조회하면 국토부 API-OpenAI API 호출을 생략하고 캐시된 결과를 즉시 반환하도록 모듈 레벨 캐시를 추가하였다. 응답 JSON에 "cached": true/false 를 포함해 어떤 경로로 응답했는지 확인할 수 있다.

[그림 11] 캐시 동작 검증 — 1차 호출(cached:false) → 2차 호출(cached:true)

```
문제 출력 디버그 콘솔 터미널 포트
PS C:\Users\user\Desktop\A1-3> curl.exe "http://localhost:3000/api/analyze?gu=%EC%84%B1%EC%82%B0%EA%B5%AC"
{"gu": "성산구", "change": 9.8, "monthly": [{"month": "202602", "avg": 40827}, {"month": "202603", "avg": 36312}, {"month": "202604", "avg": 37229}, {"month": "202605", "avg": 42017}, {"month": "202606", "avg": 41764}, {"month": "202607", "avg": 43949}], "summary": "창원시 성산구의 최근 6개월 아파트 평균 매매가는 약 9.8% 상승했습니다. 2월에는 평균 40 027만원(약 4억 0천만원)으로 시작했으나, 7월에는 43949만원(약 4억 3천만원)으로 증가했습니다. 이번 데이터는 성산구 아파트 가격이 시간에 따라 오름세를 보았음을 나타냅니다.", "cached": false}
PS C:\Users\user\Desktop\A1-3> curl.exe "http://localhost:3000/api/analyze?gu=%EC%84%B1%EC%82%B0%EA%B5%AC"
{"gu": "성산구", "change": 9.8, "monthly": [{"month": "202602", "avg": 40827}, {"month": "202603", "avg": 36312}, {"month": "202604", "avg": 37229}, {"month": "202605", "avg": 42017}, {"month": "202606", "avg": 41764}, {"month": "202607", "avg": 43949}], "summary": "창원시 성산구의 최근 6개월 아파트 평균 매매가는 약 9.8% 상승했습니다. 2월에는 평균 40 027만원(약 4억 0천만원)으로 시작했으나, 7월에는 43949만원(약 4억 3천만원)으로 증가했습니다. 이번 데이터는 성산구 아파트 가격이 시간에 따라 오름세를 보았음을 나타냅니다.", "cached": true}
PS C:\Users\user\Desktop\A1-3> █
```

```
PS> curl.exe "http://localhost:3000/api/analyze?gu=%EC%84%B1%EC%82%B0%EA%B5%AC"
{"gu": "성산구", "change": 9.8, ..., "summary": "창원시 성산구의 최근 6개월 아파트 평균 매매가는 약 9.8% 상승했습니다. ...", "cached": false}

PS> curl.exe "http://localhost:3000/api/analyze?gu=%EC%84%B1%EC%82%B0%EA%B5%AC"
{"gu": "성산구", "change": 9.8, ..., "summary": "창원시 성산구의 최근 6개월 아파트 평균 매매가는 약 9.8% 상승했습니다. ...", "cached": true}
```

두 응답의 gu · change · monthly · summary 가 완전히 동일하면서 cached 값만 false → true 로 바뀐 것이, 캐시가 실제로 재사용되었다는 증거이다.

③ 문서 보완 (README.md)

나머지 2건(개선 계획 3·4번)은 코드 변경이 아닌 문서 보완 사항으로, README.md 에 아래 내용을 추가하였다.

- API 요청/응답 예시(JSON) 및 Mermaid 시퀀스·아키텍처 다이어그램 (README "5. 과제 수행방법" 섹션)
- 캐시 대안 비교표(Vercel KV, 경량 모델 튜닝, Cron 사전계산, HTTP 캐시 헤더) (README "16. 개선 사항" 섹션)
- API 키 유출 시 대응 절차: 폐기 → 재발급 → 재배포 → 커밋 이력 정리 (README "17. 보안 사고 대응 절차" 섹션)

3) 최종 사전평가 결과 (보완 조치 반영 후)

종합 점수	결과
100%	17개 항목 전체 통과 (PASS 17 / FAIL 0)

종합 평가: 최초 평가에서 지적된 #4(입력 검증), #16(응답 지연 개선) 모두 실제 코드로 구현하고, 로컬 환경에서 정상/경계/오류 케이스를 직접 호출해 재검증하였다. 문서화가 필요했던 나머지 항목(API 예시, 아키텍처 다이어그램, 키 유효성 대응 절차)도 README.md에 반영을 완료하여 사전평가 지적 사항을 모두 해소하였다.

4) 개선 계획 반영 현황

- [x] AI 입력에 길이 제한·유효성 검증을 추가하고, 초과 시 안내 메시지를 제공한다. (#4 보완 — 코드 구현 및 재검증 완료)
- [x] 응답 지연 개선을 위한 캐시 전략·경량 모델·요약 전처리 방안을 문서화한다. (#16 보완 — 캐시 코드 구현 및 재검증 완료, 나머지 대안은 README에 문서화)
- [x] API 요청/응답 예시(JSON)와 간단한 아키텍처·시퀀스 다이어그램을 문서에 추가한다. (README.md 반영 완료)
- [x] 키 유효성 시 대응 순서(폐기 → 재발급 → 커밋 정리)를 README에 요약한다. (README.md 반영 완료)

IX. 자체 점검 체크리스트

최종 결과물·기능요구사항·보너스·개발환경 항목을 카테고리별로 세분화하여 점검하였다.

1) 최종 결과물

점검 항목	달성
구 선택 입력 → AI 요약 출력까지 정상 동작	○
페이지/섹션 3개 이상 + 상단 네비게이션 이동	○
모바일/데스크톱 2개 이상 화면 크기에서 레이아웃 유지	○
국토부 실거래가 + OpenAI 연동 결과가 화면에 표시	○
빈 입력 / API 오류 / 지연 실패 처리 안내 제공	○

2) 기능 요구사항

점검 항목	달성
① 서비스 기획 — 목적·타겟·페이지 3개 이상, AI 기능 정의	○
② 프로젝트 구조 — index.html, css/, js/, api/, images/, requirements.txt + GitHub	○
③ 프론트엔드 — 바닐라 HTML/CSS/JS, 페이지 간 네비게이션	○
④ 반응형 — 모바일/데스크톱 2가지 이상 화면 크기 확인	○
⑤ AI UX — 입력 UI, 결과 표시, 실패 처리(빈 입력/오류/지연)	○
⑥ AI API 연동 — api/에 Python 함수, OpenAI 호출, requirements.txt	○
⑦ 배포·검증 — GitHub-Vercel 연동, 배포 URL 동작 확인, 재배포	○
⑧ 문서화 — README, 서비스 기획서, 증빙 스크린샷	○

3) 보너스 과제

점검 항목	달성
방문자 분석(Vercel Web Analytics) 적용 및 정상 동작	○
4개 페이지에 분석 스크립트 삽입, 대시보드에 조회수 기록 확인	○

4) 개발환경 · 제약사항

점검 항목	달성
프론트엔드 순수 HTML/CSS/JavaScript 사용 (프레임워크 미사용)	○
백엔드 Vercel Serverless Functions(Python) 사용	○
API 키를 환경 변수로 관리, 코드/README/스크린샷에 미노출	○
배포 URL에서 동작 재현 가능 (제3자 접속·테스트 가능)	○
템플릿/예시를 그대로 복제하지 않음 (아이디어·구성 변경)	○

X. 최종 제출물 (필수 5종)

구분	내용	상태
① 배포된 웹 서비스	changwondaek.vercel.app (4페이지, 반응형, AI 기능)	완료
② GitHub 저장소	프론트(HTML/CSS/JS) + 백엔드(api/) 구조 구분	완료
③ README.md	소개/기술스택/실행·배포/환경변수 설정법	완료
④ 서비스 기획서	docs/ 폴더 (목적/타겟/페이지/AI 입출력·실패처리)	완료
⑤ 증빙 자료	web/images/ (데스크톱·모바일·AI 동작·코딩 로그)	완료

X I. 결론 및 배운 점

본 프로젝트를 통해 다음을 직접 경험하고 설명할 수 있게 되었다.

- HTML/CSS/JavaScript의 역할 구분과, 사용자 입력이 fetch 요청으로 바뀌어 응답이 화면에 반영되는 흐름
- Vercel Serverless Functions(Python)로 프론트에서 백엔드를 호출하는 구조
- 환경 변수로 API 키를 안전하게 관리하는 이유와 방법 (.env.local, .gitignore, Vercel 환경변수)
- 로컬 환경과 배포 환경의 차이, 배포 후 발생한 문제를 직접 진단·수정·재배포하는 과정
- AI 코딩 도구가 생성한 코드라도 오류 원인을 파악하고 해결하는 능력

※ 본 서비스는 국토부 공개 실거래가 기반 참고 정보이며 투자 조언이 아니다. 최근 1~2개월 데이터는 신고 지연으로 실제와 다를 수 있다.
본 보고서는 AI Native Advanced 개인 미션(A1-3) 수행 결과를 정리한 것이다.